Project 4 Writeup

Cricket Bergner

04/23/2026

1 Introduction

Project 4 explores the formation of aggregates through a stochastic simulation of diffusion limited aggregation. To form the fractal-like aggregate, particles will be spawned in on a grid, then randomly wander until they hit and stick to another particle. Conceptually, this simulation draws attention to convergence and speed of the algorithm, as incorrect implementation could cause long run times, or extremely asymmetric fractals. The complexities of the simulation will be explored throughout the writeup.

1.1 Languages and Libraries

The code for this project was written in Python. The libraries used were matplotlib.pyplot, numpy, random, matplotlib.animation's FuncAnimation, and mpl_toolkits.mplot3d's Axes3D.

This was the first project to use the random library, and its applications were quite interesting. The random library allows for a random number to be generated within the program. This can be confined to a set of numbers (as in generating a random number between 1 and 5), or not confined. The generation of random numbers is useful due to its application of being used to simulate Monte Carlo data.

FuncAnimation is able to repeatedly call a function to update a plot, making an animation. In this case, FuncAnimation saves the updated aggregate as a frame, then plays all the frames in succession and saves it as a GIF. This allows for the aggregate's growth to be viewed in real time.

Axes3D enables 3D plotting in Matplotlib. This new library allows for plotting in three spatial dimensions.

2 Undergirding Theory

2.1 Diffusion Limited Aggregation

Diffusion Limited Aggregation (DLA) is a stochastic growth process in which particles that undergo random motion stick together to form a fractal-like aggregate. This process begins with a seed particle, and then introduces other particles. These particles randomly wander

until they hit each other, in which they either bounce off or stick together. If they stick together, the aggregate grows. Since the particle motion is through diffusion (random motion), growth is significant in the outer branches of the aggregate, leading to irregular, fractal-like patterns that vary with each run. The resulting structures are characterized by non-integer capacity dimension, since they will have gaps of empty space between the branches. Variations in the sticking probability and number of particles will affect how the aggregate forms and its density.

2.2 Capacity Dimension versus Topological Dimension

A capacity dimension measures how dense a structure is by determining how its number of covering elements scale as their size approaches zero. It is usually used in fractal geometry to estimate the dimension of irregular shapes or fractals.

A topological dimension is determined by the number of independent variables needed to represent it, regardless of the dimension of the space it is in. Think of a plane being represented by two independent vectors, even though a plane like the x-z plane can sit within the $\mathbf{R}^3$.

Thus, the difference between the two is that the topological dimension describes how many independent directions a structure can move, while capacity dimension describes how much space a structure takes up.

2.3 Witten and Sander's Published Capacity Dimension

Witten and Sander found a non-integer capacity dimension of 1.66 for a square lattice and 1.67 for a triangular lattice in 2D.

For various iterations of the code below, the capacity dimension always came out to around 1.3 or 1.4. Thus, the capacity dimension did not match Witten and Sander's results. This is likely a result of the wispy branches. Lower stickiness and higher particle count would fill in the aggregate more, leading to a higher capacity dimension. Additionally, the use of a finite grid for the simulation introduces boundary effects that may limit cluster growth. Adjusting the spawn and kill radius, along with the grid size, could also be possible fixes for this result.

2.4 How Capacity Dimension Changes as a Function of Stickiness

Here, S represents the stickiness, which can be represented as any number between [0,1]. If $S = 1$, then the particle will always stick to every other particle it hits. This can cause for strong branches in the aggregate to form, as particles will stick as soon as they touch a branch. This leaves them no opportunity to bounce and settle into less populated areas. As a result, there will be a lot of empty space in the particle, which will decrease the capacity dimension.

If $S = 0$, the particle will not stick at all. Thus, the optimal value for the DLA simulation is usually $S < 1$. However, if $S < 1$, then the particle may bounce around many times before sticking, while can increase the run time. That being said, if the particle has the opportunity

to wander, then it has a better chance of settling into less populated areas of the aggregate, which will make the aggregate more "filled out". This will increase the capacity dimension.

Thus, as S decreases, capacity dimension will increase.

3 Walking Through the Code

Before the code, all the relevant libraries are imported. The following code was split into sections for visual clarity. Each section will be discussed in depth below.

```
# import libraries
import numpy as np
from matplotlib import pyplot as plt
import random as ra
from matplotlib.animation import FuncAnimation
from mpl_toolkits.mplot3d import Axes3D
```

3.1 Section 1: Functions and Initial Variables

The first function, "calculate_capacity_dimension" estimates the capacity dimension using the box-counting method.

The code starts by finding all the occupied pixels on the grid, divides the space into boxes, then counts how many boxes contain parts of the cluster. It then makes a log-log plot of number of boxes versus box size, finds the slope of the resulting line, and returns it as a decimal value. This decimal value is the capacity dimension. The accuracy of the capacity dimension is ultimately limited by the grid size and number of particles.

```
# calculated capacity dimension for different DLA functions
def calculate_capacity_dimension(grid):
    # calculates the box-counting dimension
    pixels = np.argwhere(grid > 0)
    if len(pixels) < 2: return 0

    # Determine the maximum possible box size based on grid shape
    max_side = max(grid.shape)
    scales = np.unique(np.floor(np.logspace(0, np.log10(max_side /
        2), 15)).astype(int))
    scales = scales[scales > 1]
    ns = []

    for s in scales:
        bins = (pixels // s).astype(int)
        unique_bins = np.unique(bins, axis=0)
        ns.append(len(unique_bins))
```

```
    # linear regression
    coeffs = np.polyfit(np.log(scales), np.log(ns), 1)
    return -coeffs[0]
```

The next three functions are the movement functions for the different DLA scenarios: 2D, 2D triangular, and 3D. These functions simulate a random walk. Each particle moves one step at a time in a randomly chosen direction.

For the 2D DLA, there are four directions to move in. For the triangular 2D DLA, there are six directions, and there are six directions for the 3D DLA. The lattice structure defined by these functions change how the particles explore space. This will impact the resulting shape of the aggregate.

```
# for 2D DLA, particle wandering
def move_2D(px, py):
    direction = ra.randint(0, 3)
    if direction == 0: py += 1
    elif direction == 1: py -= 1
    elif direction == 2: px -= 1
    elif direction == 3: px += 1
    return px, py

# for 2D Triangular DLA, particle wandering
def move_2Dt(px, py):
    direction = ra.randint(0, 5)
    if direction == 0: px += 1
    elif direction == 1: px -= 1
    elif direction == 2: py += 1
    elif direction == 3: py -= 1
    elif direction == 4:
        py += 1
        px += (1 if py % 2 != 0 else -1)
    elif direction == 5:
        py -= 1
        px += (1 if py % 2 != 0 else -1)
    return px, py

# for 3D DLA, particle wandering
def move_3D(px, py, pz):
    d = ra.randint(0, 5)
    if d == 0: px += 1
    elif d == 1: px -= 1
    elif d == 2: py += 1
    elif d == 3: py -= 1
    elif d == 4: pz += 1
    elif d == 5: pz -= 1
```

```
    return px, py, pz
```

The sticking functions determine whether a particle attaches to the cluster when it comes near it. Essentially, the function will check the particle's neighboring cells and calls in the stickiness variable if another particle is nearby (a number between 0 and 1, representing the probability that a function will stick). If the random variable is within the probability, the two particles will merge. If not, then the particle will bounce off and continue to wander.

Stickiness is what introduces stochastic growth. Higher stickiness results in more branching, as particles will not wander as far down the cluster as possible. Lower stickiness will likely result in denser clusters. Since the particle will bounce around more, it had a greater chance of settling into the less populated areas of the aggregate.

```
# if particle nears another particle, it sticks
def sticking(px, py, grid, stickiness):
    if np.any(grid[px-1:px+2, py-1:py+2] > 0):
        return ra.random() <= stickiness
    return False

# if particle nears another particle, it sticks (DLA 2D triangular)
def sticking_2Dt(px, py, grid, stickiness, n):
    dx = 1 if py % 2 != 0 else -1
    neighbors = [(px+1, py), (px-1, py), (px, py+1), (px, py-1),
                 (px+dx, py+1), (px+dx, py-1)]
    for i, j in neighbors:
        if 0 <= i < n and 0 <= j < n and grid[i, j] > 0:
            return ra.random() <= stickiness
    return False
```

The "update_2D" function is the cornerstone of making the animations of the aggregates growing. It updates the frames for the result GIF files every set number of particles (50 was used for 2D and 2D triangular, 20 was used for 3D). This allows it to let the animation run smoother and reduces compiling time. Similar functions were defined for the 2D triangular and 3D extensions.

First, the function calls one frame in the animation. It then processes the set number of particles per frame (here it is 50). A particle is placed randomly on the circle of radius "spawn" around the cluster. The particle walks around until it sticks to another particle, or wanders too far and hits the kill radius. The function then loops to keep updating frames with the resulting aggregate that forms.

```
# saves a frame every 50 particles for the 2D DLA animation
def update_2D(frame):
    global spawn, r_max

    # run the simulation 50 times per frame
    particles_per_frame = 50
```

```
    for _ in range(particles_per_frame):
        theta = ra.uniform(0, 2*np.pi)
        px, py = int(center + spawn*np.cos(theta)), int(center +
            spawn*np.sin(theta))

        while True:
            px, py = move_2D(px, py)

            # kill radius check
            dist = np.sqrt((px-center)**2 + (py-center)**2)
            if dist > (spawn * kill_radius): break
            if px < 1 or px >= n-1 or py < 1 or py >= n-1: break

            if sticking(px, py, grid_ani_2d, stickiness):
                grid_ani_2d[px, py] = 1
                if dist > r_max:
                    r_max = dist
                    spawn = r_max + 10
                break # particle stuck, move to next particle

    im2d.set_array(grid_ani_2d)
    ax.set_title(f"2D DLA : Particles={frame}, S={stickiness}")
    return [im2d]
```

With all that being defined, the initial variables were set. The assignment specified that 5000 particles were used. The grid was set to 250 by 250 to allow for a larger aggregate to form, and the stickiness was varied throughout the runs to create bushier or stick-like fractals.

```
# initial variables
n = 250
grid_2d = np.zeros((n, n))
center = n // 2
grid_2d[center, center] = 1
spawn, r_max = 10, 0
num_particles = 5000
stickiness = 0.6
kill_radius = 4
```

3.2 Main Body: DLA 2D

This section performs a DLA on a 2D square lattice.

3.2.1 Section 2: Plot

The process is simple in concept. A particle is spawned on a circle. It will then call on the move function to wander around. If it sees another particle as a "neighbor", the sticking

function will determine if it sticks or bounces off. If the particle wanders too far without sticking, it will hit the kill radius and be removed. As the cluster grows, the spawn radius is updated. This ensures particles are introduced near the edges of the cluster, which makes them more likely to stick. When a particle wanders for too long without sticking, it will cause the program to run longer.

```
print("")
print("Generating 2D DLA Plot...")

for i in range(num_particles):
    theta = ra.uniform(0, 2*np.pi)
    px, py = int(center + spawn*np.cos(theta)), int(center +
        spawn*np.sin(theta))
    while True:
        px, py = move_2D(px, py)
        dist = np.sqrt((px - center)**2 + (py - center)**2)

        # if too far, stop this particle
        if dist > (spawn * kill_radius):
            break
        if px < 1 or px >= n-1 or py < 1 or py >= n-1: break
        if sticking(px, py, grid_2d, stickiness):
            grid_2d[px, py] = 1
            dist = np.sqrt((px-center)**2 + (py-center)**2)
            if dist > r_max:
                r_max = dist
                spawn = r_max + 10
            break

print("")
plt.figure(figsize=(6,6))
plt.imshow(grid_2d, cmap='magma')
plt.title(f"2D DLA : Particles={num_particles}, S={stickiness}")
plt.savefig("dla_2d.png", dpi=300)
plt.show()
```

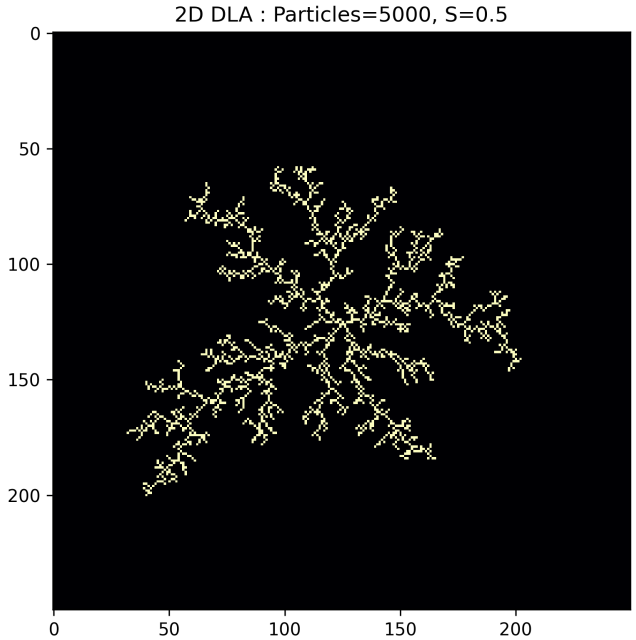

Figure 1: The resulting 2D aggregate from the code above. Stickiness is set to 0.5, there are 5000 particles, and the grid is 250 by 250. The lower stickiness factor allows for the branches to grow in density as the particles wander lower on the aggregate.

3.2.2 Section 3: Animation

This section maps the growth of the 2D DLA aggregate into a GIF animation. It calls upon the update functions to load the frames. The code will then save the animation as a GIF file to the computer running it.

```
print("Generating 2D DLA Animation...")
grid_ani_2d = np.zeros((n, n))
grid_ani_2d[center, center] = 1
spawn, r_max = 10, 0

fig, ax = plt.subplots(figsize=(6,6))
im2d = ax.imshow(grid_ani_2d, cmap='magma', animated=True)

save_on_this_particle = 50
ani2d = FuncAnimation(fig, update_2d, frames=range(0, num_particles,
    save_on_this_particle), interval=50, blit=False)
ani2d.save('dla_2d_animation.gif', writer='pillow')
plt.close()
```

3.3 Extension 2: DLA Triangular 2D

The process here is similar to the DLA 2D square lattice. A particle is introduced, wanders around, and either sticks or is eventually killed off. What makes the triangular lattice different is that each point has six neighbors, allowing for diagonal movement in addition to the horizontal and vertical movement. If there are more movement directions, then there will be greater diffusion. Theoretically, the cluster should grow in a more balanced way, as the particles can approach it from more angles.

3.3.1 Section 4: Plot

Here, a particle is introduced within the spawn circle. It will then move in six possible directions, and call the triangular sticking function to see if it should stick to a neighbor or keep wandering. Once all 5000 particles have followed this process, the result is plotted.

```
print("")
print("Generating Triangular DLA Plot...")
grid_tri = np.zeros((n, n))
grid_tri[center, center] = 1
spawn, r_max = 10, 0

for i in range(num_particles):
    theta = ra.uniform(0, 2*np.pi)
    px, py = int(center + spawn*np.cos(theta)), int(center +
        spawn*np.sin(theta))
    while True:
        px, py = move_2Dt(px, py)
        if px < 1 or px >= n-1 or py < 1 or py >= n-1: break
        if sticking_2Dt(px, py, grid_tri, stickiness, n):
            grid_tri[px, py] = 1
            dist = np.sqrt((px-center)**2 + (py-center)**2)
            if dist > r_max:
                r_max = dist
                spawn = r_max + 10
            break

print("")
plt.figure(figsize=(6,6))
plt.imshow(grid_tri, cmap='magma')
plt.title(f"Triangular DLA : Particles={num_particles}, S=
    {stickiness}")
plt.savefig("dla_triangular.png", dpi=300)
plt.show()
```

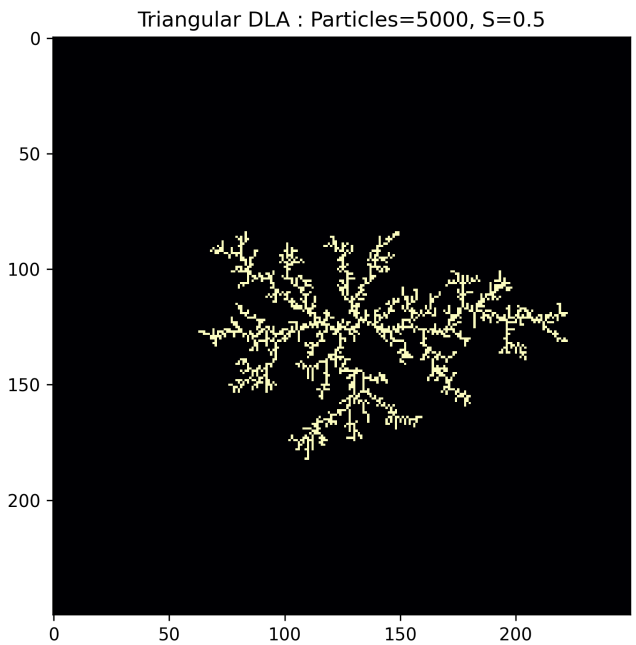

Figure 2: An aggregate formed by the code above for the DLA 2D triangular lattice. It was made with 5000 particles and a stickiness factor of 0.5 on a 250 by 250 grid.

3.3.2 Section 5: Animation

Similar to what was described in Section 3, large batches of particles will be mapped to animation frames as they diffuse via the DLA triangular lattice. These frames will be compiled into an animation, which the code will save as a file to the computer running it.

```
print("")
print("Generating Triangular DLA Animation...")
grid_ani_tri = np.zeros((n, n))
grid_ani_tri[center, center] = 1
spawn, r_max = 10, 0

fig, ax = plt.subplots(figsize=(6,6))
im_tri = ax.imshow(grid_ani_tri, cmap='magma', animated=True)

# Update the animation call to use the sub-sampling range
ani_tri = FuncAnimation(fig, update_tri, frames=range(0,
    num_particles, 50), interval=50, blit=False)
ani_tri.save('dla_triangular_animation.gif', writer='pillow')
plt.close()
```

3.4 Extension 3: DLA 3D

The 3D cube lattice works the same as the other two in terms of overall motion. Like with the triangular lattice, the 3D DLA can move in six directions, however, this time it is because a third dimension is introduced. The particles will diffuse in volume, not just a plane. This increases the opportunities for wandering and decreases sticking.

3.4.1 Section 6: Plot

Here, 3D plotting needs to be introduced to be able to plot the 3D aggregate and motion. Since an extra dimension is added, rendering is slower and the resulting figure may be harder to interpret, as it will be seen in 2D. However, the process mirrors the ones before it. A particle is introduced and wanders until it sticks or is killed. The program plots the resulting aggregate.

```
print("")
print("Generating 3D DLA Plot...")
n3 = 80
grid_3d = np.zeros((n3, n3, n3))
center3 = n3 // 2
grid_3d[center3, center3, center3] = 1
spawn3 = 5

for i in range(num_particles - 4000):
    phi = ra.uniform(0, 2*np.pi)
    cost = ra.uniform(-1, 1)
    theta = np.arccos(cost)
    px = int(center3 + spawn3 * np.sin(theta) * np.cos(phi))
    py = int(center3 + spawn3 * np.sin(theta) * np.sin(phi))
    pz = int(center3 + spawn3 * np.cos(theta))

    while True:
        px, py, pz = move_3D(px, py, pz)
        dist = np.sqrt((px-center3)**2 + (py-center3)**2 + (pz-
            center3)**2)
        if dist > (spawn3 * kill_radius):
            break
        if px < 1 or px >= n3-1 or py < 1 or py >= n3-1 or pz < 1 or
            pz >= n3-1: break
        if np.any(grid_3d[px-1:px+2, py-1:py+2, pz-1:pz+2] > 0):
            grid_3d[px, py, pz] = 1
            dist = np.sqrt((px-center3)**2 + (py-center3)**2 + (pz-
                center3)**2)
            if dist > spawn3 - 2: spawn3 = dist + 5
            break
```

```
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
z, y, x = np.nonzero(grid_3d)
ax.scatter(x, y, z, s=1, c=z, cmap='magma')
plt.title(f"3D DLA : Particles={num_particles - 4000}, S={stickiness}")
plt.savefig("dla_3d.png", dpi=300)
plt.show()
```

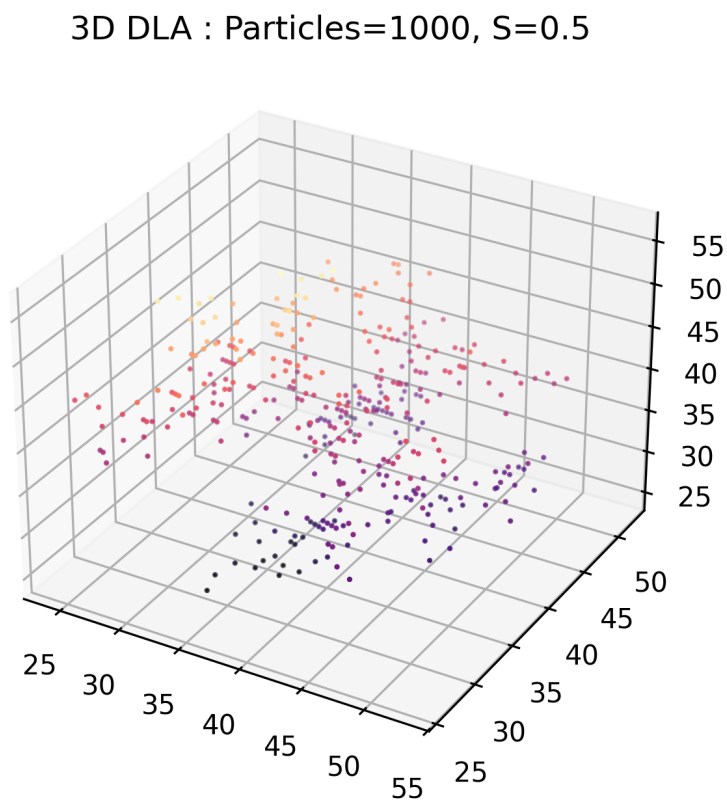

Figure 3: The 3D DLA cubic lattice aggregate formed by the code above. It has 1000 particles as compared to the other two, as the extra dimension impacts the run time of the code significantly. The stickiness factor is 0.5.

3.4.2 Section 7: Animation

For every set of 20 particles, the aggregate is mapped to a frame. The frames are compiled into an animation that is saved as a GIF file.

```
print("Generating 3D DLA Animation...")
grid_ani_3d = np.zeros((n3, n3, n3))
```

```
grid_ani_3d[center3, center3, center3] = 1
spawn3 = 5

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

save_on_this_particle_3D = 20
ani3d = FuncAnimation(fig, update_3d, frames=range(0, num_particles
    - 4000, save_on_this_particle_3D), interval=100, blit=False)
ani3d.save('dla_3d_animation.gif', writer='pillow')
plt.close()
```

3.5 Section 8: Capacity Dimension

This section calls on the capacity dimension function to calculate the capacity dimension for each of the three aggregates, so they can be compared. The 3D lattice should have a higher capacity dimension as compared to the 2D simulations, as the extra dimension allows for denser clusters.

```
print("")
print("Final Capacity Dimensions")
print("-" * 30)
dimension_2d = calculate_capacity_dimension(grid_2d)
dimension_tri = calculate_capacity_dimension(grid_tri)
dimension_3d = calculate_capacity_dimension(grid_3d)

print(f"Capacity Dimension (2D):    {dimension_2d:.4f}")
print(f"Capacity Dimension (2D Triangular):  {dimension_tri:.4f}")
print(f"Capacity Dimension (3D):    {dimension_3d:.4f}")
print("-" * 30)
print("")
```

One of the runs of this code produced the following output. The values do not agree with the expected value, but as explained in Section 2.3, this number can change with the number of particles, stickiness factor, and allowed grid size.

Capacity Dimension (2D): 1.4563
Capacity Dimension (2D Triangular): 1.3891
Capacity Dimension (3D): 1.3933

4 Lessons Learned

This project gave me a much better conceptual understanding of code run times and how they are impacted. Little things like reducing the kill radius, or increasing stickiness could take the algorithm from running for an hour to running for a few minutes. Sometimes coding feels like following a set of steps, but I am gaining a better appreciation for the conceptual

side of this process. I will also say I learned a lot about animation in Python, between figuring out what libraries to use, to how to implement it and to how to reduce frames to compile the code faster. It was neat to be able to make my own GIFs, and I am sure it will be a useful skill for projects in the future.

I will say I did a pretty bad job at keeping a linear workflow. By that I mean I had three different drafts of the code stored on Google Colab, Python, and GitHub throughout this whole process. Each code file would get different tweaks, so I could keep track of what edits worked, and which ones did not. While I was able to keep this process consistent in my head, and mentally track which file worked the best, I realize that it left my workflow very hard to track through GitHub commits alone, since the code changed substantially throughout the process. For the final project I will attempt to be more linear with my workflow, and keep only one file going throughout the process, without having so many test files.

4.1 Time Keeping

The code took around twenty-five hours to write, between the 100 particle first draft, adding the extensions, implementing the animations, calculating capacity dimension, adding functions to decrease runtime and make more filled out aggregates, and all the research that went into implementation. The writeup took around six hours. Thus, I would say around forty to fifty hours went into this project total, between classes, discussions of the assignment, research, implementation, and run times of the code.

4.2 Additional Notes/Thoughts

I ran into some difficulties putting all the pieces of the assignment together. Originally I had developed a solid algorithm that made nice, rounded aggregates with a function called "get_balanced_angle". This function mapped out angular slices of the big circle that was the grid. It would calculate which circle slice had the fewest particles, then spawn a new one there. It was a nice function that I think improved my output quite a bit. However, I had to remove it when I started implementing the animations. When I got the animations to start working, and tried to add "get_balanced_angle" back in, the animations started failing. I sadly do not have a good enough understanding of the code to figure out what went wrong. Call it a combination of there being so much code to look at and my struggle this semester to realize coding is a deeply conceptual process. While I am glad my final code does what it is supposed to, I do lament the function I had to leave behind.

```
# pick angle where particles are not spawning
def get_balanced_angle(counts, num_bins):
    # lower count = higher weight
    weights = 1.0 / (counts + 0.1)
    probabilities = weights / np.sum(weights)
    bin_idx = np.random.choice(range(num_bins), p=probabilities)
    bin_width = (2 * np.pi) / num_bins
    return ra.uniform(bin_idx * bin_width, (bin_idx + 1) * bin_width)
```

4.3 Attributions

- https://www.sciencedirect.com/topics/engineering/capacity-dimension - A brief overview on capacity dimension.
- https://www.sciencedirect.com/topics/mathematics/topological-dimension - A brief overview on topological dimension.
- https://paulbourke.net/fractals/dla/ - A great visualization on how stickiness affects the density of the aggregate.
- https://www.dictionary.com - I would like to sound more technical, and part of that is using big words! My favorite big word for this writeup is "stochastic", meaning "of or relating to a process involving a randomly determined sequence of observations each of which is considered as a sample of one element from a probability distribution".
- https://pmc.polytechnique.fr/pagesperso/dg/cours/biblio/PRB%2027,%205686%20(1983)%20Witten,%20Sander%20%5BDiffusion-limited%20aggregation%5D.pdf - Witten and Sander's paper on DLA and capacity dimension.
- https://stackoverflow.com/questions/47882315/funcanimation-increase-fps-for-long-an - Increasing speed of animations. Round 1's 5K particle animations were incredibly slow because every frame was shown.
- https://stackoverflow.com/questions/53697293/how-to-plot-data-with-axes3d-in-python - Examples of how to use the Axes3D library.
- https://www.geeksforgeeks.org/python/matplotlib-animation-funcanimation-class-in-py - In depth explanation of the FuncAnimation library.
- Michael explained the process of implementing animations in Python to me, and advised me on what libraries to use. He also helped me see that running the code in Google Colab does not always load the animations.
- Emma's roommate Mia (a computer science major) helped me figure out how the run the python code through my terminal in VSCode, which admittedly I was having trouble figuring out how to do.